

AVALIAÇÃO PRÁTICA – INTERNET DAS COISAS

Curso Técnico em Desenvolvimento de Sistemas

Unidade Curricular: Internet das Coisas

Turma: T DESI 2024/1 N1

Docente: Fernando Costenaro Silva

Data: 18/11/2025

INTEGRANTES DA EQUIPE

- Joabe Costa
- Luiz Filipe
- Abilio Francisco
- Wallace Oliveira

DESCRIÇÃO DO PROJETO

Este projeto consiste no desenvolvimento de um **microserviço IoT** para coleta e disponibilização de dados de sensores via protocolo HTTP. O serviço foi desenvolvido em **Node.js com Express**, containerizado com **Docker** e hospedado em uma instância **AWS EC2**.

Objetivo

Criar um serviço web capaz de:

- Receber dados de sensores (temperatura e status do ar-condicionado) via requisição POST
- Armazenar o último valor recebido em memória
- Disponibilizar consulta ao último valor via requisição GET
- Executar de forma isolada em container Docker na nuvem AWS

Contexto

O projeto simula um cenário real de **modernização de infraestrutura industrial**, migrando sistemas legados para uma arquitetura baseada em microserviços, comunicação via APIs RESTful e deploy em containers na nuvem.

ESTRUTURA DO CÓDIGO

Arquitetura do Projeto

```
avaliacao-iot-service/
├── Dockerfile           # Configuração do container
├── .dockerignore        # Arquivos ignorados no build
├── package.json         # Dependências Node.js
└── requests.http        # Testes REST Client
```

```
|— src/
|   |— server.js          # Servidor Express (rotas e lógica)
|   |— docs/
|       |— ENTREGA.md      # Documento de entrega
|       |— Screenshots/    # Prints dos testes
```

Dockerfile Comentado

```
# Dockerfile para avaliacao-iot-service
FROM node:20-alpine

# Diretório de trabalho dentro do container
WORKDIR /app

# Copia apenas arquivos de dependências primeiro (melhor cache)
COPY package*.json ./

# Instala apenas produção (imagem menor). Usamos npm install pois não há lockfile.
RUN npm install --omit=dev

# Copia o restante do código
COPY . .

# Expõe a porta do serviço
EXPOSE 3000

# Comando de inicialização
CMD ["npm", "start"]
```

Explicação linha a linha:

- `FROM node:20-alpine`: Usa imagem oficial do Node.js 20 na versão Alpine (apenas 40MB, otimizada)
- `WORKDIR /app`: Define `/app` como diretório padrão para execução dos comandos
- `COPY package*.json ./`: Copia arquivos de dependências antes do código (aproveita cache do Docker)
- `RUN npm install --omit=dev`: Instala apenas dependências de produção (não instala nodemon e ferramentas de dev)
- `COPY . .`: Copia todo o código-fonte restante
- `EXPOSE 3000`: Documenta que o container escuta na porta 3000
- `CMD ["npm", "start"]`: Executa `node src/server.js` ao iniciar o container

server.js – Código Completo

```
const express = require('express');

const app = express();
const PORT = process.env.PORT || 3000;
```

```
let lastData = null;

app.use(express.json());

app.get('/', (req, res) => {
  res.json({
    status: 'ok',
    service: 'avaliacao-iot-service',
    routes: ['/api/registro (POST)', '/api/ultimo (GET)']
  });
});

app.post('/api/registro', (req, res) => {
  const { temperatura, ar } = req.body || {};

  if (typeof temperatura !== 'number' || typeof ar !== 'boolean') {
    return res.status(400).json({
      ok: false,
      message: 'Payload inválido. Envie {"temperatura": number, "ar": boolean}'
    });
  }

  lastData = { temperatura, ar, recebidoEm: new Date().toISOString() };

  return res.status(201).json({ ok: true, recebido: lastData });
});

app.get('/api/ultimo', (req, res) => {
  if (!lastData) {
    return res.status(404).json({
      ok: false,
      message: 'Nenhum dado registrado ainda.'
    });
  }
  return res.json({ ok: true, ultimo: lastData });
});

app.listen(PORT, () => {
  console.log(`Servidor iniciado em http://0.0.0.0:${PORT}`);
});
```

Estrutura do código:

- **Inicialização:** Cria servidor Express na porta 3000
- **Middleware:** `express.json()` faz parsing automático de JSON no body das requisições
- **Armazenamento:** Variável `lastData` guarda em memória o último valor recebido
- **Validação:** Verifica tipos de dados antes de armazenar (temperatura numérica, ar booleano)

EXPLICAÇÃO DA API

Rotas Implementadas

1 GET /

Função: Healthcheck e documentação básica

Resposta: 200 OK

```
{
  "status": "ok",
  "service": "avaliacao-iot-service",
  "routes": ["/api/registro (POST)", "/api/ultimo (GET)"]
}
```

2 POST /api/registro

Função: Receber e armazenar dados de sensores

Content-Type: application/json

Body esperado:

```
{
  "temperatura": 20.3,
  "ar": true
}
```

Validações:

- `temperatura` deve ser do tipo `number`
- `ar` deve ser do tipo `boolean`
- Retorna **400 Bad Request** se validação falhar

Resposta de sucesso (201 Created):

```
{
  "ok": true,
  "recebido": {
    "temperatura": 20.3,
    "ar": true,
    "recebidoEm": "2025-11-19T01:30:45.123Z"
  }
}
```

3 GET /api/ultimo

Função: Consultar o último valor registrado

Resposta (200 OK) quando há dados:

```
{
  "ok": true,
  "ultimo": {
    "temperatura": 20.3,
    "ar": true,
    "recebidoEm": "2025-11-19T01:30:45.123Z"
  }
}
```

Resposta (404 Not Found) quando vazio:

```
{
  "ok": false,
  "message": "Nenhum dado registrado ainda."
}
```

Códigos de Status HTTP Utilizados

Código	Significado	Uso na API
200	OK	GET bem-sucedido com dados
201	Created	POST bem-sucedido, recurso criado
400	Bad Request	Dados inválidos no POST
404	Not Found	GET quando não há dados armazenados

 PRINTS ORGANIZADOS

1. Execução Local (PowerShell)

Build da imagem Docker localmente

```

PS C:\Users\joabe_s_costa\Documents\SA-4Fase> cd .\avaliacao-iot-service\
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker build -t avaliacao-iot-service .
[+] Building 2.1s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 514B
=> [internal] load metadata for docker.io/library/node:20-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 129B
=> [1/5] FROM docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435
=> => resolve docker.io/library/node:20-alpine@sha256:6178e78b972f79c335df281f4b7674a2d85071aae2af020ffa39f0a770265435
=> [internal] load build context
=> => transferring context: 233B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --omit=dev
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:23bcc33bce17228731ba166bd971427263c5fbb454bdb3120040a02fa4852f84
=> => exporting config sha256:505e4f549cb60de05132481e9cd330c07a80fded90bde288499f825bf3c3547a
=> => exporting attestation manifest sha256:11e408833a382329595d9777c4c491445fee4b5e3985f1f9a55c6d4cdc1abeab
=> => exporting manifest list sha256:072e16f1d2b9963319969f1d9d3b6d20ae20edd1f4e22bfa530c37718015d037
=> => naming to docker.io/library/avaliacao-iot-service:latest
=> => unpacking to docker.io/library/avaliacao-iot-service:latest

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/w40710qdd3hfc5k1k0tj0d5q5
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker run -d -p 3001:3000 --name avai-test avaliacao-iot-service
bd59d29cab265fef2b8ee35be64d28469ccd2c64c1b791722d2d4442d579c4ac
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker logs -f avai-test

> avaliacao-iot-service@1.0.0 start
> node src/server.js

Servidor iniciado em http://0.0.0.0:3000
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker stop avai-test
avai-test
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker rm avai-test
avai-test
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker login
Authenticating with existing credentials... [Username: joabecost]

Info → To Login with a different account, run 'docker Logout' followed by 'docker Login'

Login Succeeded
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker tag avaliacao-iot-service:latest joabecost/avaliacao-iot-service:latest
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker push joabecost/avaliacao-iot-service:latest
The push refers to repository [docker.io/joabecost/avaliacao-iot-service]
f6451dec4d62: Waiting
4ca8f3070275: Waiting
e74e4ed823e9: Waiting
14b42b7dc7b8: Waiting

PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker push joabecost/avaliacao-iot-service:latest
e74e4ed823e9: Waiting
14b42b7dc7b8: Waiting
2d35ebdb57d9: Waiting
da04d522c98f: Waiting
4cfc7c7dc731: Waiting
60e45a9660cf: Waiting
dc898e71499b: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker push joabecost/avaliacao-iot-service:latest
The push refers to repository [docker.io/joabecost/avaliacao-iot-service]
e74e4ed823e9: Waiting
14b42b7dc7b8: Waiting
4cfc7c7dc731: Waiting
2d35ebdb57d9: Waiting
da04d522c98f: Waiting
dc898e71499b: Waiting
60e45a9660cf: Waiting
f6451dec4d62: Waiting
4ca8f3070275: Waiting
push access denied, repository does not exist or may require authorization: server message: insufficient_scope: authorization failed
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: CLDR-QPZM
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...
docker info
Login Succeeded
PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker info
Client:
 Version:      28.4.0
 Context:      desktop-linux
 Debug Mode:   false
 Plugins:

```

```
ai: Docker AI Agent - Ask Gordon (Docker Inc.)  
  Version: v1.9.11  
  Path: C:\Program Files\Docker\cli-plugins\docker-ai.exe  
buildx: Docker Buildx (Docker Inc.)  
  Version: v0.28.0-desktop.1  
  Path: C:\Program Files\Docker\cli-plugins\docker-buildx.exe  
cloud: Docker Cloud (Docker Inc.)  
  Version: v0.4.27  
  Path: C:\Program Files\Docker\cli-plugins\docker-cloud.exe  
compose: Docker Compose (Docker Inc.)  
  Version: v2.39.2-desktop.1  
  Path: C:\Program Files\Docker\cli-plugins\docker-compose.exe  
debug: Get a shell into any image or container (Docker Inc.)  
  Version: 0.0.42  
  Path: C:\Program Files\Docker\cli-plugins\docker-debug.exe
```

```

desktop: Docker Desktop commands (Docker Inc.)
  Version: v0.2.0
  Path: C:\Program Files\Docker\cli-plugins\docker-desktop.exe
extension: Manages Docker extensions (Docker Inc.)
  Version: v0.2.31
  Path: C:\Program Files\Docker\cli-plugins\docker-extension.exe
init: Creates Docker-related starter files for your project (Docker Inc.)
  Version: v1.4.0
  Path: C:\Program Files\Docker\cli-plugins\docker-init.exe
mcp: Docker MCP Plugin (Docker Inc.)
  Version: v0.18.0
  Path: C:\Program Files\Docker\cli-plugins\docker-mcp.exe
model: Docker Model Runner (Docker Inc.)
  Version: v0.1.40
  Path: C:\Program Files\Docker\cli-plugins\docker-model.exe
sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
  Version: 0.6.0
  Path: C:\Program Files\Docker\cli-plugins\docker-sbom.exe
scout: Docker Scout (Docker Inc.)
  Version: v1.18.3
  Path: C:\Program Files\Docker\cli-plugins\docker-scout.exe

Server:
Containers: 0
Running: 0
Paused: 0
Stopped: 0
Images: 2
Server Version: 28.4.0
Storage Driver: overlayfs
  driver-type: io.containerd.snapshotter.v1
Logging Driver: json-file
Cgroup Driver: cgroupfs
Cgroup Version: 2
Plugins:
  Volume: local
  Network: bridge host ipvlan macvlan null overlay
  Log: awslogs fluentd gcplogs gelf journald json-file local splunk syslog
CDI spec directories:
  /etc/cdi
  /var/run/cdi
Discovered Devices:
  cdi: docker.com/gpu=webgpu
Swarm: inactive
Runtimes: runc io.containerd.runc.v2 nvidia
Default Runtime: runc
Init Binary: docker-init
containerd version: 05044ec0a9a75232cad458027ca83437aae3f4da
runc version: v1.2.5-0-g59923ef
init version: de40ad0
Security Options:
  seccomp
  Profile: builtin
cgroupns

```

```

Kernel Version: 6.6.87.2-microsoft-standard-WSL2
Operating System: Docker Desktop
OSType: linux
Architecture: x86_64
CPUs: 20
Total Memory: 7.601GiB
Name: docker-desktop
ID: 7557d245-fdc6-43d6-af2b-7a97b2c3c165
Docker Root Dir: /var/lib/docker
Debug Mode: false
HTTP Proxy: http.docker.internal:3128
HTTPS Proxy: http.docker.internal:3128
No Proxy: hubproxy.docker.internal
Labels:
  com.docker.desktop.address=npipe://\\.\pipe\docker_cli
Experimental: false
Insecure Registries:
  hubproxy.docker.internal:5555
  ::1/128
  127.0.0.0/8
Live Restore Enabled: false

PS C:\Users\joabe_s_costa\Documents\SA-4Fase\avaliacao-iot-service> docker push joabecost/avaliacao-iot-service:latest
The push refers to repository [docker.io/joabecost/avaliacao-iot-service]
2d35ebdb57d9: Pushed
60e45a9660cf: Pushed
dc898e71499b: Pushed
da04d522c98f: Pushed
4cfc7c7dc731: Pushed
f6451dec4d62: Pushed
4ca8f3070275: Pushed
e74e4ed823e9: Pushed
14b42b7dc7b8: Pushed
latest: digest: sha256:072e16f1d2b9963319969f1d9d3b6d20ae20edd1f4e22bfa530c37718015d037 size: 856

```

Comandos executados:

```
# Build da imagem
docker build -t avaliacao-iot-service .

# Executar container localmente
docker run --name avaliacao-iot -p 3000:3000 -d avaliacao-iot-service

# Verificar logs
docker logs -f avaliacao-iot
```

2. Push para Docker Hub

Comandos executados:

```
# Tag da imagem
docker tag avaliacao-iot-service joabecost/avaliacao-iot-service:latest

# Login no Docker Hub
docker login

# Push da imagem
docker push joabecost/avaliacao-iot-service:latest
```

Imagem disponível em: `joabecost/avaliacao-iot-service:latest`

3. Execução na AWS EC2

Terminal da EC2 – Pull e execução

Comandos executados na EC2:

Configuração da EC2

aws academy

Conta

Painel de controle

Cursos

Calendário

Caixa de entrada

Histórico

Ajuda

Página inicial

Módulos

Fóruns

Lucid (Quadro branco)

ALLv2PT-BR-LTI13-130129 > Módulos > Laboratório de aprendizagem da AWS Academy > Iniciar os laboratórios de aprendizagem da AWS Academy

AWS Used \$0.8 of \$50 02:30 ▶ Start Lab ■ End Lab ⓘ AWS Details ⓘ Readme ↺ Reset ✕

eee_0L5250659@runweb1969431 ~\$

PT-BR

Learner Lab

[Visão geral do ambiente](#)
[Navegação pelo ambiente](#)
[Acessar o console de gerenciamento da AWS](#)
[Restrição de Regiões](#)
[Uso do serviço e outras restrições](#)
[Usar o terminal no navegador](#)
[Executar os comandos da AWS CLI](#)
[Usar o AWS SDK for Python](#)
[Preservar seu orçamento](#)
[Acessar instâncias do EC2](#)
[Acesso por SSH a instâncias do EC2](#)
[Acesso por SSH pelo Windows](#)
[Acesso por SSH pelo Mac](#)

Data da última atualização das instruções: 24/06/2025

Visão geral do ambiente

Este Learner Lab oferece um ambiente de sandbox para a exploração específica de serviços da AWS.

Este ambiente é de longa duração. Quando o cronômetro da sessão for zerado, a sessão terminará,

◀ Anterior

Próximo ▶

✓ Prova Prática IoT i-095c561824da29a34 Executando 🔍 t3.small 3/3 verificações a Exibir alarmes + us-east-1f ec2-44-197-115-40.co... 44.197.115.40 -

Resumo da instância para i-095c561824da29a34 (Prova Prática IoT) Informações

Atualizado há about 1 hour

ID da instância

i-095c561824da29a34

Endereço IPv6

-

Tipo de nome do host

Nome do IP: ip-172-31-76-141.ec2.internal

Nome do DNS do recurso privado de resposta

IPv4 (A)

Endereço IP atribuído automaticamente

44.197.115.40 [IP público]

Função do IAM

-

IMDSv2

Required

Operador

-

Endereço IPv4 público

44.197.115.40 | [endereço aberto](#)

Estado da instância

Executando

Nome do DNS de IP privado (somente IPv4)

ip-172-31-76-141.ec2.internal

Tipo de instância

t3.small

ID da VPC

vpc-0aa8bc96d74d30ece

ID da sub-rede

subnet-00e8bd28f4f3948b7

ARN da instância

arn:aws:ec2:us-east-1:794881392957:instance/i-095c561824da29a34

Endereços IPv4 privados

172.31.76.141

DNS pública

ec2-44-197-115-40.compute-1.amazonaws.com | [endereço aberto](#)

Endereços IP elásticos

-

Descoberta do AWS Compute Optimizer

[Opte por participar do AWS Compute Optimizer para obter recomendações.](#) | [Saiba mais](#)

Nome do Grupo do Auto Scaling

-

Gerenciado

falso

Configurações aplicadas:

- **Instância:** t2.micro (Free Tier)
- **AMI:** Amazon Linux 2023
- **Security Group:** Porta 80 (HTTP) aberta para 0.0.0.0/0
- **IP Público:** 44.197.115.40

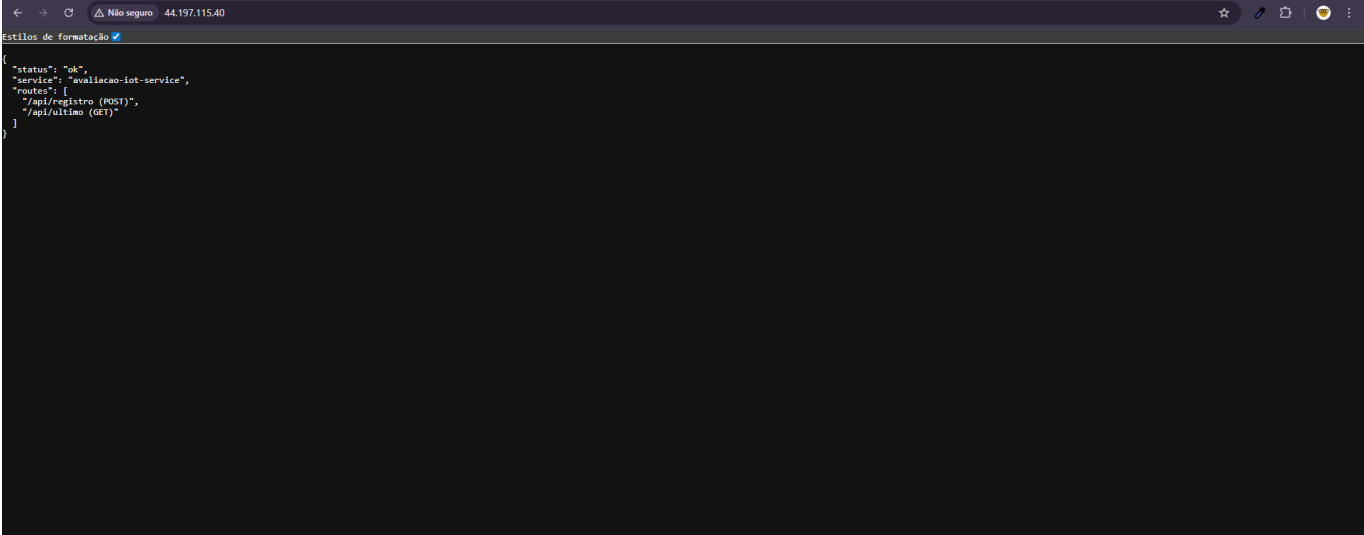
4. Testes via Thunder Client

Ferramenta utilizada: Thunder Client (extensão do VS Code)

Motivo da escolha: Interface visual intuitiva, integração nativa com o ambiente de desenvolvimento e facilidade para salvar/exportar requisições.

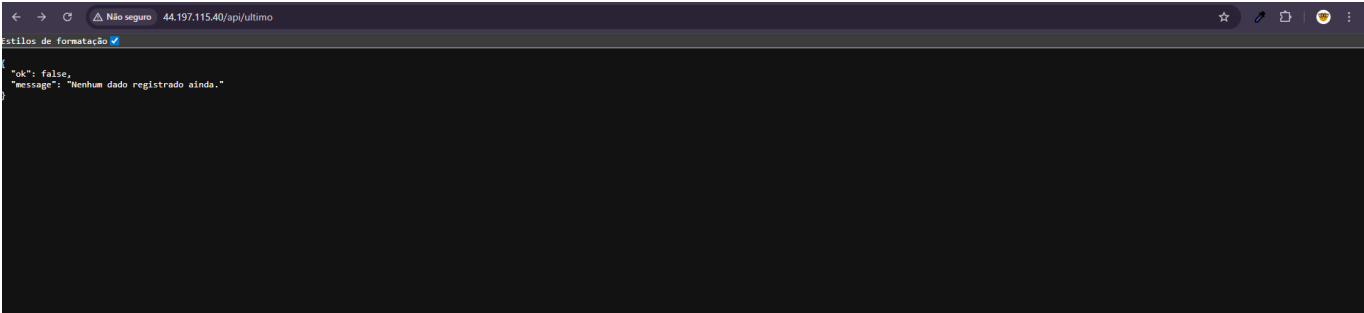
GET / – Healthcheck

11 / 15



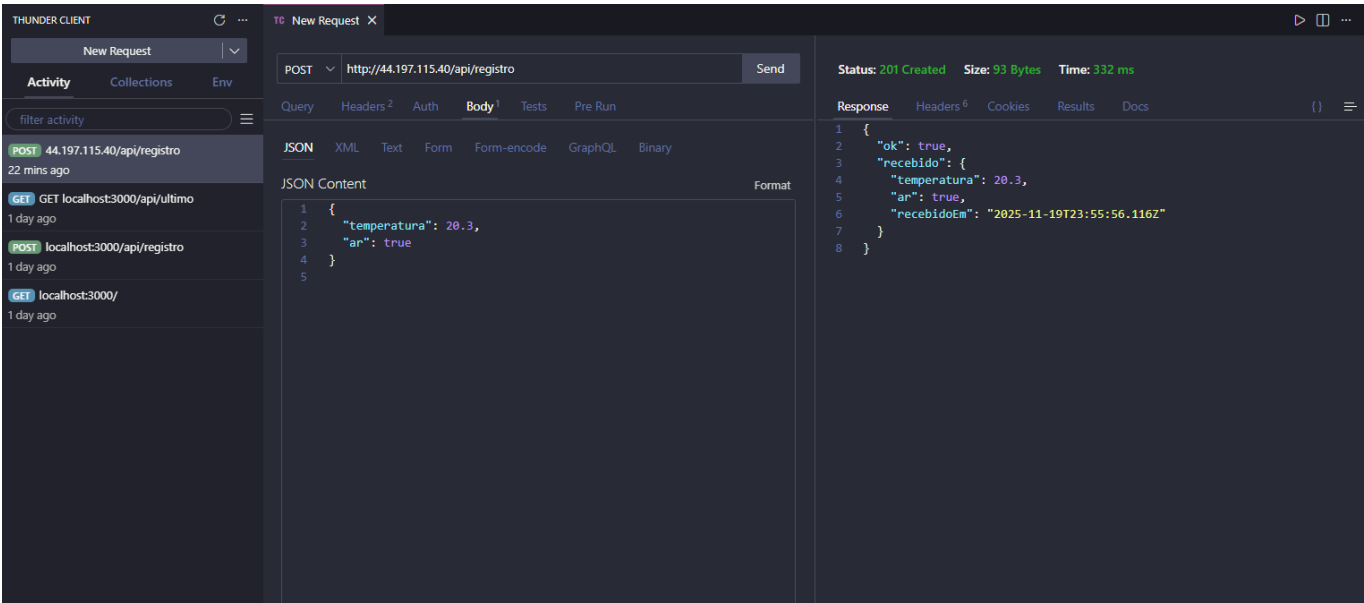
Resultado: Status 200 OK, retornando informações do serviço e rotas disponíveis.

GET /api/ultimo – Antes de qualquer POST



Resultado: Status 404 Not Found, mensagem "Nenhum dado registrado ainda" (comportamento esperado).

POST /api/registro – Envio de dados do sensor

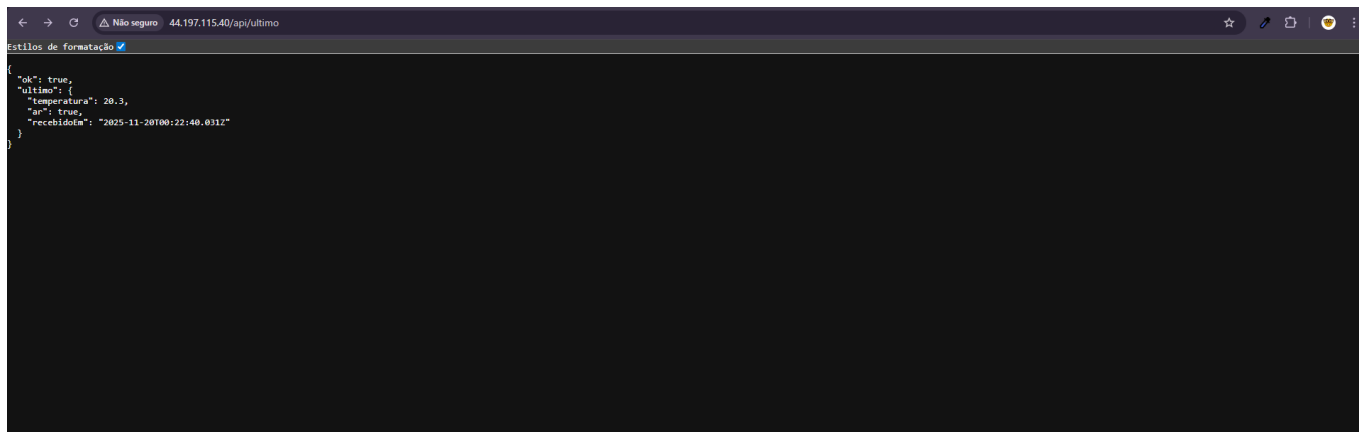


Body enviado:

```
{
  "temperatura": 20.3,
  "ar": true
}
```

Resultado: Status 201 Created, confirmação do recebimento com timestamp.

GET /api/ultimo – Após o POST



Resultado: Status 200 OK, retornando o último valor registrado com temperatura 20.3, ar true e timestamp.

TECNOLOGIAS E CONCEITOS UTILIZADOS

Protocolo HTTP

- **GET:** Método para leitura de recursos (idempotente, não altera estado)
- **POST:** Método para envio de dados e criação de recursos
- **Headers:** **Content-Type:** `application/json` indica formato JSON no body
- **Status Codes:**
 - 2xx (sucesso): 200 OK, 201 Created
 - 4xx (erro do cliente): 400 Bad Request, 404 Not Found
 - 5xx (erro do servidor): não implementado neste projeto (servidor estável)

JSON (JavaScript Object Notation)

JSON (JavaScript Object Notation) é um formato leve de intercâmbio de dados, legível por humanos e facilmente interpretado por máquinas. Baseado em um subconjunto da linguagem JavaScript, é amplamente utilizado em APIs RESTful para estruturar dados em pares chave-valor.

Características:

- Formato texto (string)
- Suporta tipos: string, number, boolean, array, object, null
- Independente de linguagem
- Sintaxe simples: `{"chave": "valor"}`

Exemplo usado no projeto:

```
{
  "temperatura": 20.3,
  "ar": true
}
```

Uso na API:

- O cliente (Thunder Client) envia JSON no body do POST
- O servidor parseia com `express.json()` middleware
- A resposta também é JSON (padrão REST)

Docker

- **Container:** Ambiente isolado que empacota aplicação + dependências
- **Imagem:** Template read-only usado para criar containers
- **Dockerfile:** Receita para construir a imagem
- **Docker Hub:** Registry público para compartilhar imagens

Benefícios:

- Portabilidade (roda igual em qualquer ambiente)
- Isolamento (não conflita com outras aplicações)
- Eficiência (containers são mais leves que VMs)

Node.js + Express

- **Node.js:** Runtime JavaScript server-side, assíncrono e eficiente
- **Express:** Framework web minimalista para criação de APIs REST
- **Middleware:** Funções que processam requisições (ex: `express.json()`)

AWS EC2 (Elastic Compute Cloud)

- **Instância:** Servidor virtual na nuvem AWS
- **Security Group:** Firewall virtual para controlar tráfego de entrada/saída
- **IP Público:** Endereço acessível pela internet

**FUNCIONAMENTO DAS ROTAS****Fluxo de dados – POST /api/registro**

```
Cliente (Thunder Client)
  ↓ POST /api/registro
  ↓ Body: {"temperatura": 20.3, "ar": true}
  ↓
Express Middleware (express.json())
  ↓ Parseia JSON → req.body
```

```
↓
Validação (server.js)
  ↓ Verifica tipos de temperatura e ar
  ↓
Armazenamento (variável lastData)
  ↓ lastData = { temperatura, ar, recebidoEm }
  ↓
Resposta 201 Created
  ↓ {"ok": true, "recebido": {...}}
  ↓
Cliente recebe confirmação
```

Fluxo de dados – GET /api/ultimo

```
Cliente (Thunder Client)
  ↓ GET /api/ultimo
  ↓
Verificação (server.js)
  ↓ lastData === null ?
  ↓
Se vazio:
  → Resposta 404 Not Found
  → {"ok": false, "message": "Nenhum dado registrado ainda."}

Se preenchido:
  → Resposta 200 OK
  → {"ok": true, "ultimo": {...}}
```

REFERÊNCIAS

- Express.js Documentation: <https://expressjs.com/>
- Docker Official Documentation: <https://docs.docker.com/>
- Node.js Documentation: <https://nodejs.org/docs/>
- AWS EC2 User Guide: <https://docs.aws.amazon.com/ec2/>
- HTTP Status Codes (MDN): <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>
- REST API Best Practices: <https://restfulapi.net/>
- JSON Specification: <https://www.json.org/>
- Thunder Client (VS Code): <https://www.thunderclient.com/>

Documento elaborado em: 19/11/2025

Repositório GitHub: <https://github.com/LuizFli/SA-4Fase>

Imagem Docker Hub: joabecost/avaliacao-iot-service:latest

Endpoint AWS: <http://44.197.115.40>